

Liquidity Launcher Audit



Uniswap

August 21, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	4
Scope	5
Phase 1	5
Phase 2	5
Phase 3	6
Phase 4	6
System Overview	8
Security Model and Trust Assumptions	11
High Severity	15
H-01 Fees Attributed to CompoundingClaimRecipient Can Be Permanently Locked by Saturating a Boundary Tick	15
Medium Severity	16
M-01 Native ETH Held by FeeSplitter Can Be Stolen via a Self-Declared Pool Creator	16
M-02 Positions Merely Approved to FeeSplitter Can Have Fees Diverted to Fallback Recipients by Any Caller	17
M-03 Creator Registration Coupled To Payout Permanently Locks Unregistered Position Fees	18
Low Severity	19
L-01 Launch Position Is Minted Out of Range, Leaving the Pool Without Active Liquidity at Initialization	19
L-02 FeeSplitter Accepts Non-Native-ETH PositionManager Positions and Permanently Bricks Fee Collection	20
L-03 Hardcoded Empty hookData Can Permanently Brick Fee Collection for Hook-Restricted Positions	21
L-04 Documented ERC-165 Callback Gate Does Not Exist	22
L-05 Positions Launched Through a Forwarder Can Have Their Beneficiary NFT Claimed by Anyone	22
L-06 Unverified External Return Value Authorizes Beneficiary NFT Claims	24
L-07 Oversized Graffiti Response Permanently Blocks Claims For Unregistered Positions	25
L-08 Prefunded Router Inputs Can Be Stranded and Swept by Third Parties	26
Notes & Additional Information	27
N-01 Unsanitized creator() Results Can Strand or Burn the CREATOR Share	27
N-02 Incorrect and Incomplete Documentation	28
N-03 Floating Pragma	28
N-04 Unused Import	29
N-05 Functions Updating State Without Event Emissions	29
N-06 Missing Docstrings	29
N-07 Incomplete Docstrings	30
N-08 Coding Inconsistencies	31
N-09 Unused Named Return Variables	32

Client Reported	33
CR-01 Permissionless Pool Initialization Allows Permanently Bricking a Direct Launch	33
CR-02 Creator Fee Share Silently Misdirected to LiquidityLauncher on Every Launch	34
Conclusion	36
Appendix	38
Issue Classification	38

Summary

Type	DeFi	Total Issues	23 (11 resolved)
Timeline	From 2026-07-22 To 2026-07-28	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	1 (1 resolved)
		Medium Severity Issues	3 (1 resolved)
		Low Severity Issues	8 (1 resolved)
		Notes & Additional Information	9 (7 resolved)
		Client Reported Issues	2 (1 resolved)

Scope

OpenZeppelin performed an audit of the [Uniswap/liquidity-launcher](#) repository over four phases.

Phase 1

The first phase initially reviewed commit [100fe70](#) and, following changes made in response to that review, concluded with commit [607e3fa](#).

In scope were the following files:

```
src
├── periphery
│   └── FeeSplitter.sol
└── strategies
    └── DirectLaunchStrategy.sol
```

Phase 2

The second phase reviewed the reworked fee-distribution design at commit [2812203](#).

In scope were the following files:

```
src
├── interfaces
│   ├── IBeneficiaryVault.sol
│   ├── IClaimExecutor.sol
│   ├── IClaimableRecipient.sol
│   └── IFeeSplitter.sol
├── periphery
│   ├── BaseClaimRecipient.sol
│   ├── BaseClaimRecipientWithCallback.sol
│   ├── BeneficiaryVault.sol
│   ├── CompoundingClaimRecipient.sol
│   └── FeeSplitter.sol
└── strategies
    └── DirectLaunchStrategy.sol
```

Phase 3

The third phase reviewed a scope extension that adds a UERC20-specific beneficiary vault, building on the same periphery fee-claim framework as the previous phases. The review covered the changes introduced in pull request [#211](#) at commit [be73d32](#).

In scope were the following files:

```
src
├── interfaces
│   ├── external
│   │   └── IUERC20.sol
│   └── IUERC20BeneficiaryVault.sol
├── periphery
│   ├── BeneficiaryVault.sol
│   └── UERC20BeneficiaryVault.sol
└── strategies
    └── InstantLaunchStrategy.sol
```

Phase 4

The fourth phase took a one day and reviewed a scope extension that adds a Universal Router strategy, the native distribution path on the launcher that funds it, and a rework of UERC20 beneficiary registration. The review covered the changes introduced in [PR226](#) between commit [ef47c07](#) and commit [e3acb7a](#).

In scope were the following files:

```
src
├── interfaces
│   ├── external
│   │   ├── IUniversalRouter.sol
│   │   ├── IBeneficiaryVault.sol
│   │   ├── ILiquidityLauncher.sol
│   │   ├── IMulticall.sol
│   │   ├── INativeStrategy.sol
│   │   └── IPermit2Forwarder.sol
│   └── periphery
│       ├── BaseClaimRecipient.sol
│       ├── BeneficiaryVault.sol
│       ├── ProtocolFeeController.sol
│       └── UERC20BeneficiaryVault.sol
├── strategies
│   └── UniversalRouterStrategy.sol
├── LiquidityLauncher.sol
├── Multicall.sol
└── Permit2Forwarder.sol
```

`IUERC20BeneficiaryVault.sol` was removed in this phase and its error inlined into `UERC20BeneficiaryVault`.

System Overview

Liquidity Launcher is a token launch system built on Uniswap v4 that supports token creation, distribution, and liquidity bootstrapping. This engagement covers the direct launch path of that system and the periphery that receives the fees it generates. The first phase reviewed `DirectLaunchStrategy` and `FeeSplitter` in a design where the splitter resolved a launch's fee beneficiary itself, through a sentinel entry in its splits and a beneficiary registered on the splitter at the moment the position was deposited. The second phase reviewed the design that replaced it, in which the splitter holds no beneficiary state at all and per-position attribution is handled by a separate family of claim recipients. The third phase reviewed a scope extension to that design: `DirectLaunchStrategy` was renamed to `InstantLaunchStrategy` and its coupling to the beneficiary vault made optional, and a new `BeneficiaryVault` variant, `UERC20BeneficiaryVault`, was added that lets the creator of a launcher-created token claim an unregistered position's beneficiary share by proving authorship through the token's on-chain graffiti. The fourth phase reviewed a further extension that adds a strategy running caller-supplied Universal Router routes, the native distribution path on the launcher that funds it, and a rework of how UERC20 beneficiary registration is authorized and how often it may occur. The descriptions below reflect the implementation in scope at each phase. All phases rely on the canonical Uniswap v4 `PositionManager` and `PoolManager`, which are outside the scope of this audit.

DirectLaunchStrategy: Launches a fixed-supply token into a hookless, native-ETH pool, and is driven only by the configured `LiquidityLauncher` through its `initializeDistribution` entry point. Every pool parameter is fixed at deployment: a one-billion-unit supply of an 18-decimal token, a 25-basis-point fee, a single tick spacing, and a single price band. As a result, one deployed instance launches every token at the same opening price. The strategy pulls the full supply from the launcher (rejecting fee-on-transfer and rebasing tokens through a balance-difference check), initializes a native-ETH pool in which the token is the non-base currency, and mints a single-sided position covering the range from the minimum usable tick up to the opening tick. All supply therefore sits on the token side of the price, so the pool opens with only the launched token available. Because the position's upper boundary is the opening tick itself, the position is out of range at the exact tick the pool is initialized to, and buyers bring it into range as the pool tick moves down toward the position's lower boundary. While it still holds the position, the strategy registers the launch configuration's

fee beneficiary with the `BeneficiaryVault`, and only afterwards transfers the position to the `FeeSplitter` for permanent custody.

FeeSplitter: A singleton, immutable-configuration custodian of native-ETH v4 liquidity positions. Its configuration is a fixed list of recipients, each carrying an independent native ETH share and token share in basis points, with each side summing to the full denominator. Any address can call `collectFees` for a set of positions. For each one, the contract realizes the accrued fees through a zero-liquidity decrease and then pushes every recipient its share of the contract's full standing balance on both sides. Native transfers are force-sent, so a recipient that cannot accept plain ETH cannot block a collection, while a token transfer that reverts still blocks it. Recipients flagged as callback recipients are notified through `onAmountsReceived` immediately after their shares are pushed, so they can attribute the amounts received to the position they came from. A notification that reverts causes the whole collection call to revert. A separate permissionless `increaseLiquidity` entry point adds liquidity to a custodied position from funds already held by the `PositionManager`, returning any excess to the caller, and requires the position to carry no uncollected fees. The splitter's receiver callback rejects any NFT that does not come from the `PositionManager` and ignores the accompanying transfer data, but it is not the only route by which a position arrives: a plain transfer, as performed by `DirectLaunchStrategy`, bypasses the callback entirely, so no property of an inbound position is validated at deposit. No code path transfers or approves a position back out, so custody is permanent and fee distribution stays permissionless for the life of the position.

Claim recipients: A family of contracts that sit behind the splitter's splits and turn a pushed fee share into a claimable balance. `BaseClaimRecipient` records the amounts announced for each position, admitting them only when the contract's balance covers everything already attributed, and exposes a `claim` entry point guarded by per-side minimum amounts. A transfer policy hook decides, for each claim, who is paid and how much, and attribution is decremented before any funds move. `BaseClaimRecipientWithCallback` extends that base so that every claim calls back into the claimer through `onClaimed`, bracketed by hooks that let the recipient verify that the claimer met an obligation across the callback.

`CompoundingClaimRecipient` uses the bracket to snapshot the position's liquidity and require the claimer to have raised it by at least a configured minimum, so a claim succeeds only if the claimed fees are compounded back into the position. `BeneficiaryVault` inherits the plain base instead, and represents a position's beneficiary share as a transferable ERC-721 minted at registration. Only the current holder of that token may claim, and a position with no registration pays its share out to immutable per-side fallback addresses.

Beneficiary registration: Registration with the vault is authorized by proving custody of the position in the `PositionManager`, which is why `DirectLaunchStrategy` registers while it

still holds the freshly minted position. Re-registration replaces the beneficiary, and any unclaimed credit follows the beneficiary token rather than the address that first received it. Once the position reaches the splitter, no address can satisfy the custody proof again, so from that point the beneficiary of a launched position can change only by transferring the beneficiary token.

InstantLaunchStrategy: The third phase's rename of `DirectLaunchStrategy`; the two are the same contract, and its launch mechanics are unchanged from the description above. The functional change reviewed in this phase makes the strategy's coupling to the beneficiary machinery optional. The vault supplied at deployment may now be the zero address, which opts the instance out of creator fees and leaves every launch position unregistered, and the strategy registers a launch's fee beneficiary only when a vault is configured. Correspondingly, the deploy-time check that the configured vault is wired into the splitter as a fees-callback recipient has been removed. Everything else, including the rejection of the zero address and the launcher as fee beneficiaries, is retained.

UERC20BeneficiaryVault: A `BeneficiaryVault` variant, reviewed in the third phase, that adds a second way to authorize a claim, aimed at positions that were never registered — for example, those launched by strategies that predate the vault. It overrides the base's claim-transfer hook so that, the first time an unregistered position is claimed, the vault reads the on-chain graffiti of each side of the pool through a defensive `staticcall` to `IUERC20.graffiti()` that treats any failure or ill-formed return as absent graffiti. Launcher-created tokens record the address that called the `LiquidityLauncher` as their graffiti, so a caller whose `keccak256(abi.encode(msg.sender))` matches either side's graffiti is treated as the creator and is minted the position's beneficiary token, after which every later claim runs the base's ordinary owner check. If a side carries graffiti but none matches the caller, the claim reverts with `NotTokenCreator`; if neither side carries graffiti — a non-launcher token, or the native-ETH side — the vault falls back to the base behavior and pays the unregistered position's shares to the immutable per-side fallback addresses. Because the first successful proof mints a transferable NFT, the graffiti route retires itself per position, and ownership can never fall back to the zero address again.

UniversalRouterStrategy: Runs one caller-supplied Universal Router route so that a launch and a buy settle in a single transaction. Both entry points are launcher-only and reentrancy-guarded, and the strategy holds no balance between calls, having no `receive`. The router is caller input rather than immutable state, so one deployment serves every router version, and the route travels as `abi.encode(commands, inputs, deadline)` decoded into a typed `execute` call. It is funded either by native forwarded to `initializeWithNative` or by an allowance on the distributed token in `initializeDistribution`, which pays the router directly so that a creator can buy with a token they already hold without approving the router.

Cleanup is the route's responsibility: output must be taken to an explicit recipient, because `TAKE_ALL` pays the router's locker, which on this path is the strategy.

Native distribution: `distributeWithNative` forwards a caller-named `nativeAmount` to an `INativeStrategy`, reported through the existing `TokenDistributed` event with `address(0)` as the token. `multicall` and the remaining entry points became `payable` so that a value-bearing batch can carry native through the launcher's self-delegatecall; only `distributeWithNative` spends it, and the launcher exposes no `receive` and no refund path. The buy rides the launcher's own `multicall`, which preserves `msg.sender`, so `createToken`'s graffiti continues to name the creator rather than an aggregator.

Beneficiary registration (fourth phase): Registration became one-shot: `registerBeneficiary` reverts with `TokenAlreadyExists` once a position's beneficiary token exists, where re-registration previously burned and re-minted it, and no burn path remains, so the beneficiary changes only by transferring the token. In `UERC20BeneficiaryVault`, the graffiti route moved from the claim-transfer hook into a public `registerBeneficiary` override authorized by position custody or a graffiti match on either currency, with `NotAuthorized` replacing `NotTokenCreator`. The hook now delegates to that override, so an unregistered position whose pool carries graffiti registers its claimer on the first claim, while one without still flushes to the immutable fallbacks. `IUERC20BeneficiaryVault` was removed and its error inlined.

Security Model and Trust Assumptions

LiquidityLauncher (the configured launcher): `DirectLaunchStrategy` accepts distributions only from the single launcher address set at deployment, so launches must route through the launcher's distribution flow. The launcher is expected to batch token acquisition and distribution atomically, since it is not designed to hold balances between transactions. A fee beneficiary set to the launcher itself would have no path to retrieve what it receives, because the launcher exposes no function that moves out an arbitrary token or native balance. For that reason, the strategy rejects the launcher as a fee beneficiary. The launcher's own implementation is outside the scope of this audit.

Launch configurator and fee beneficiary: The caller that drives a launch supplies the launch configuration naming the fee beneficiary. The beneficiary is freely chosen and carries no claim

of authorship over the launched token; it only determines who receives the vault's beneficiary token for that position, and therefore who can claim the vault's share of that position's fees. The strategy rejects the zero address and the launcher as beneficiaries, and the vault additionally rejects itself at registration. Because the beneficiary token is transferable, the right to a position's fee share can be moved or sold after the launch, and whoever holds it at claim time is paid.

UERC20 graffiti and token creators: `UERC20BeneficiaryVault` treats a launched token's graffiti as an authenticated record of its creator, and this is trusted to hold only for tokens deployed through the `UERC20Factory` and `LiquidityLauncher`, which set the graffiti to the address that called the launcher. The vault reads it through a `staticcall` that treats a revert, a non-contract, or an ill-formed return as absent graffiti, so any token that exposes a matching `graffiti()` selector returning an attacker-chosen value could present itself as created by an address of the attacker's choosing; the fallback addresses, and any position with a genuinely registered beneficiary, are unaffected because the graffiti route is consulted only for unregistered positions. Where token creation is routed through an aggregator, the graffiti records that aggregator rather than the end user, so anyone able to route a call through the aggregator can satisfy the creator proof. This authorization path is only meaningful for the launched token's own side of the pool, since the paired side is always native ETH and carries no graffiti.

Fee collection and liquidity callers: Collection is permissionless. Any address can trigger collection and distribution for any set of positions, and any address can increase the liquidity of a custodied position from funds already held by the `PositionManager`. Distribution always follows the immutable splits, so an arbitrary caller controls only the timing of a collection, not its destination. Attribution on the claim recipients is likewise permissionless and backed only by the recipient's balance, so any surplus a recipient holds beyond what is already attributed, including direct donations, can be attributed by the first caller to any position of that caller's choosing. Beyond these entry points, the launch pool itself carries no hook, so both its initialization and any subsequent liquidity modification are open at the `PoolManager` level. Its pool key is fully determined by the strategy's fixed parameters together with the launched token's address. Arbitrary accounts can therefore act on a launch pool directly, without going through the strategy or the splitter.

Claim executors: Claiming from a callback recipient requires the caller to implement `IClaimExecutor`, because the callback is unconditional and is determined by which base the recipient inherits rather than by any runtime interface probe. Such recipients pay out before invoking the callback and rely on the bracketing post-condition to enforce the claimer's obligation, so a claimer is not trusted by identity but is required to leave the position in the expected state by the end of the call. `CompoundingClaimRecipient` depends in particular

on the splitter's `increaseLiquidity` path remaining usable, since that is the only route through which a custodied position's liquidity can grow.

Split recipients and the launched token: A callback recipient whose notification reverts causes the whole collection call to revert. A token-side transfer that reverts has no equivalent of the native force-send, so it blocks collection for every position in the batch. Token creators, who control the launched token's transfer logic, and the operators of any contract configured as a split recipient are assumed not to block fee distribution in either way.

Deployment configuration: The fee splits, the vault's fallback addresses, the compounding minimum, and `DirectLaunchStrategy`'s price band and supply are all fixed through immutable state set at deployment. Every pool parameter of a `DirectLaunchStrategy` instance is fixed this way, so a single instance launches all tokens at one predetermined price band and supply. The full supply must fit in one position at that band; otherwise, deployment reverts. The strategy checks at deployment that the splitter is bound to the same `PositionManager` as itself, because a mismatch would leave every launch position's fees permanently uncollectable. It also checks that the configured vault appears among the splitter's splits as a callback recipient, because otherwise every launch's beneficiary share would go unattributed. The vault's own `PositionManager` binding is not covered by those checks, so the deployer is assumed to bind the strategy, the splitter, and the vault to the same instance. The third-phase rename of this strategy, `InstantLaunchStrategy`, relaxes two of these deploy-time guarantees: its beneficiary vault is optional, so a zero vault disables creator fees and leaves every launch position unregistered, and it no longer verifies that a configured vault is wired into the splitter as a fees-callback recipient. The deployer is therefore assumed to pair a configured vault with a splitter that actually announces its shares through the callback, because otherwise every launch's beneficiary share would go unattributed. Both fallback addresses are assumed to be able to accept their respective currencies at all times, since the payouts to them are not themselves guarded against failure. The configured split array is assumed to be small enough that iterating every entry during a collection stays within a block's gas limit. Because none of this state can be changed after deployment, a misconfiguration cannot be corrected later.

Route authors and the Universal Router: The router address and the route are unvalidated caller input, and `UniversalRouterStrategy` makes exactly one `execute` call without inspecting either. Whether swap output reaches its intended recipient and whether unspent input and native are swept out of the router therefore rest entirely on the route, and the Universal Router itself is outside the scope of this audit. Native forwarded to the strategy cannot be returned to the launcher, which has no `receive`.

Launcher balances (fourth phase): Several of the launcher's entry points accept native they never spend, and the launcher provides no refund or recovery path. Native left on the launcher is spendable by any caller through `distributeWithNative`, so the amounts forwarded within a batch are assumed to add up to the value sent.

Uniswap v4 contracts: The strategy and the periphery depend on the canonical v4 `PositionManager` and `PoolManager` for pool initialization, position minting, fee realization, liquidity accounting, and pool key resolution, and treat them as trusted external systems.

High Severity

H-01 Fees Attributed to `CompoundingClaimRecipient` Can Be Permanently Locked by Saturating a Boundary Tick

Every launch performed by the `DirectLaunchStrategy` contract mints the entire token supply into a single liquidity position whose `lower boundary is the minimum usable tick` and whose upper boundary is the configured `initialTick`. The position is then transferred to the `FeeSplitter` contract for permanent custody. The pool carries no hook, so `PoolManager.modifyLiquidity` is unrestricted. Furthermore, the permissionless `increaseLiquidity` function of the `FeeSplitter` contract is the only path through which a custodied position's liquidity can grow, and the `CompoundingClaimRecipient` contract depends on it: its `claim post-condition` reverts unless the caller raises the position's liquidity by at least `MIN_LIQUIDITY_INCREASE`.

Uniswap v4 tracks `liquidityGross` per tick as the sum of the liquidity of every position referencing that tick, irrespective of owner, and enforces a `maxLiquidityPerTick` ceiling on it whenever liquidity is added. The token amount required to hold a given liquidity at a tick scales with the square root of the price at that tick. Therefore, the minimum usable tick is the cheapest tick in the pool to saturate. Any account can pin it at the ceiling for approximately 0.0000019 tokens out of the 1,000,000,000 token supply and no ETH, after which every liquidity increase on the launch position reverts with `TickLiquidityOverflow`. Fee collection and distribution remain functional, since the ceiling is enforced only when liquidity is added. However, when the splitter's splits include a `CompoundingClaimRecipient`, claiming can never complete. Claiming is the only path by which funds leave that recipient, so the fees already attributed to it are frozen permanently. The splits are immutable, the recipient exposes no recovery function, and the splitter never transfers or approves the position out, so its liquidity cannot be reduced to create headroom. No mitigation is available to the affected parties, while the cost to the attacker amounts to little more than transaction gas. The condition is permanent and needs to be established only once per launched pool.

Note that the implicit full-range fallback position resolved by the `PositionPlanner` library carries both of its boundaries at the extreme usable ticks. It is therefore exposed in the same manner, for every strategy, at a comparable cost on either side.

Consider moving the launch position's lower boundary off the minimum usable tick, to a tick at which exhausting the per-tick liquidity allowance would require more of the launched token than will ever exist. Because the supply is fixed, this renders the attack unfundable at any price rather than merely expensive. The cost of saturating a tick depends on that tick alone and not on the opening price. Therefore a single boundary value suffices for every launch, and the constructor can verify the property at deployment.

Update: Resolved in pull requests [#215](#) and [#220](#).

Medium Severity

M-01 Native ETH Held by `FeeSplitter` Can Be Stolen via a Self-Declared Pool Creator

`FeeSplitter` is a permissionless singleton that collects fees from any native-ETH-paired Uniswap v4 position sent to it and forwards them to preconfigured recipients. In `_collect`, the native amount to distribute is read as the contract's entire current balance, `address(this).balance`, rather than an amount scoped to the position being collected. That balance is then split according to `nativeSplits`, one entry of which is typically the `CREATOR_SENTINEL`. `_resolveCreator` resolves this entry, for each call, to whatever address the position's own token returns from a plain, unauthenticated `creator()` call.

An attacker can exploit this with a token under their own control whose `creator()` returns their address, paired with native ETH in a freshly initialized pool. A position in that pool needs only the smallest possible non-zero liquidity, so it never generates any real fees, and it is then sent directly to `FeeSplitter`. Calling the permissionless `collectFees` function on that worthless position still distributes the contract's entire native balance, so the attacker receives the share configured for `CREATOR_SENTINEL` out of native ETH that was never owed to them, for example, ETH mistakenly sent directly to the contract.

Consider scoping the amount distributed by `collectFees` to the balance delta realized from decreasing the liquidity of that specific position, rather than to the contract's total balance at call time.

This issue belongs to the first phase of the audit and was identified in the `100fe70a28de1648e51b1a9793be7c0fdd3ae2d7` commit version.

Update: Acknowledged, not resolved. The team stated:

The splitter holds no standing balance by design: each collect distributes exactly what it just realized, and the only value persisting between calls is per-split rounding floor dust (a few wei per recipient per side), which is deliberately flushed on the next distribution. Extracting anything meaningful requires ETH to be sent to the contract outside the collect flow, which no protocol path does. The residual case — ETH sent to the splitter by mistake — is flushed through the configured splits on the next collect regardless of which position triggers it; the fixed recipients receive their configured bps either way, and no position's own fees are diverted.

M-02 Positions Merely Approved to FeeSplitter Can Have Fees Diverted to Fallback Recipients by Any Caller

`collectFees` is permissionless: any caller may pass an arbitrary list of `tokenId`s, and `_collect` realizes and withdraws each position's fees into the splitter through a zero-liquidity `DECREASE_LIQUIDITY` followed by `TAKE_PAIR`, with `FeeSplitter` itself as the caller of `positionManager.modifyLiquidities`.

The v4-periphery `PositionManager` only requires the caller to be the position's owner or an approved operator; it does not require the splitter to actually hold the NFT.

`feeBeneficiary[tokenId]` is populated only inside `onERC721Received`, when a position is safe-transferred in with non-empty data. A position that is merely approved to the splitter through `approve` or `setApprovalForAll`, without ever being transferred, never populates this mapping. In that case, `_collect`'s [beneficiary resolution](#) falls back to `nativeFallback` and `tokenFallback` on both currency sides. Consequently, any third party can call `collectFees` with the `tokenId` of a position an owner has merely approved, for example, ahead of an intended migration or through a stale approval left in place, and repeatedly sweep that position's accrued fees to the splitter's fixed fallback recipients instead of to the owner, for as long as the approval remains outstanding.

Consider requiring that the splitter actually own the position before collecting, either by verifying that `IERC721(address(positionManager)).ownerOf(tokenId)` returns the splitter's own address, by requiring that `feeBeneficiary[tokenId]` has been registered, or both.

This issue belongs to the first phase of the audit and was identified in the `607e3faa432fd505083452f7a2b3682fbfa8a7be` commit version.

Update: Acknowledged, not resolved. The team stated:

The premise does not hold for any position this system creates. DirectLaunchStrategy.initializeDistribution mints the launch position and transfers it to the splitter within the same call, so a launch position is never in an approved-but-not-owned state and accrues no fees before custody. The finding requires a third party to approve the splitter without transferring, which no flow asks for: the splitter never calls transferFrom or approve on a position, so an approval grants it nothing it uses. In that scenario the fees still land on the splitter's own immutable recipients and unregistered positions resolve to the vault fallbacks, so there is no attacker gain — only grieving of a self-inflicted approval.

M-03 Creator Registration Coupled To Payout Permanently Locks Unregistered Position Fees

An unregistered position paired with a launcher-created UERC20 can be claimed by the token's creator, who proves the right through the token's graffiti and receives a transferable beneficiary NFT. Because the NFT is transferable, a creator that cannot itself receive the fee currencies can move the claim right to an address that can, which is the remedy documented in [BeneficiaryVault](#).

However, that registration is performed inside `_beforeClaimTransfer`, which `claim` invokes immediately before paying the full accrued amounts to the same `msg.sender`. The mint and the transfer are therefore atomic. When the creator of record cannot accept the payout, for example a contract with no payable receive function such as Multicall3, which the instant launch tests document as the creator of record for an atomic launch and buy, `_payout` reverts with `NativeTransferFailed` and the mint is rolled back with it. Nonzero graffiti simultaneously makes the gate revert `NotTokenCreator` for every other caller, so the unregistered-position fallback flush is unreachable and the position's fees are locked with no recovery path. Registration without a transfer is possible only while nothing is attributed, since `_payout` returns early on a zero amount, but `onAmountsReceived` has no access control, so any address can attribute one wei and close that window permanently.

Consider decoupling registration from payout by adding a graffiti-proof entry point that mints the beneficiary NFT and transfers nothing, leaving the creator free to move the NFT to an address that can receive the funds before any claim is attempted.

Update: Resolved in [pull request #226](#). The team stated:

| Fixed

Low Severity

L-01 Launch Position Is Minted Out of Range, Leaving the Pool Without Active Liquidity at Initialization

In `initializeDistribution`, `DirectLaunchStrategy` opens every launch by calling `poolManager.initialize` with a fixed `initialSqrtPriceX96` derived from the immutable `initialTick`, and then defines the launch's sole position through a `PositionDefinition` with `offsetUpper` set to `0`, intending the position to span from the minimum usable tick up to the opening tick.

However, `PositionPlanner.resolveTicks` resolves `tickUpper` by adding the offset to the current tick and rounding up with `tickCeil`. Since `initialTick` is required to be an exact multiple of `TICK_SPACING`, `tickCeil` returns it unchanged, so the resolved `tickUpper` equals `initialTick` exactly. Because Uniswap v4 only counts a position's liquidity as active when `tickLower <= currentTick < tickUpper`, the minted position is out of range at the very tick the pool is initialized to. Consequently, the pool opens with `pool.liquidity == 0`, leaving a window in which the pool has no active liquidity at its advertised opening price. Depending on how `v4-core` handles zero-liquidity swaps, this could either let early callers shift the tick reported by `slot0` at negligible cost or cause swaps to revert, in both cases ahead of the first real trade.

Consider documenting this behavior so that external integrators are aware that the pool opens without active liquidity and that its initial price can be shifted at negligible cost immediately after launch.

This issue belongs to the first phase of the audit and was identified in the [100fe70a28de1648e51b1a9793be7c0fdd3ae2d7](#) commit version.

Update: Acknowledged, not resolved. The team stated:

| *Inherent to a single-sided launch, not a defect. The launch places the full token supply with zero native ETH, and an in-range position requires both currencies, so the position*

can only sit strictly on the token side of the opening price; `tickUpper == initialTick` is the only correct placement. There is no liquidity above `initialTick` in either direction, so the tick can be pushed up and back down at no cost without touching supply, and the first buy walks it back to `initialTick` for free before crossing into real liquidity. No supply or value is at risk. We will document that `slot0` is not a usable price source until the first trade crosses `initialTick`.

L-02 FeeSplitter Accepts Non-Native-ETH PositionManager Positions and Permanently Bricks Fee Collection

`FeeSplitter` is designed to permanently custody v4 `PositionManager` NFTs and permissionlessly collect their fees through `collectFees`. Because positions can never be transferred, approved, or withdrawn back out, any position the splitter ends up holding must remain compatible with its fee-collection assumptions indefinitely.

However, `onERC721Received` only checks that the caller is the canonical `PositionManager`; it never queries the deposited position's `PoolKey`. An ERC-20/ERC-20 position, whose `currency0` is not native ETH, can therefore be safe-transferred into the splitter without issue. Yet `collect` requires `poolKey.currency0` to be native ETH and reverts with `InvalidBaseCurrency` otherwise, so fee collection for such a position is permanently impossible, and the position along with any fees it accrues remains stuck forever.

Consider validating a received position's `PoolKey` inside `onERC721Received`, for example, via `positionManager.getPoolAndPositionInfo(tokenId)`, and reverting for any position whose `currency0` is not native ETH.

This issue belongs to the first phase of the audit and was identified in the `607e3faa432fd505083452f7a2b3682fbfa8a7be` commit version.

Update: Acknowledged, not resolved. The team stated:

No change. The splitter only custodies positions from `DirectLaunchStrategy`, which builds a hookless `PoolKey` by construction, so empty `hookData` is correct for every position the system creates. Accepting caller-supplied `hookData` on a permissionless `collectFees` would feed arbitrary attacker-controlled data into any hook the pool did have, which widens the surface rather than narrowing it. A deposited position whose pool needs `hookData` is uncollectable for its depositor only: `collectFees` is per-`tokenId`,

so callers exclude it and no other position is affected. This constraint is documented on the contract.

L-03 Hardcoded Empty `hookData` Can Permanently Brick Fee Collection for Hook-Restricted Positions

`FeeSplitter` is an immutable custodian of v4 position NFTs; its own documentation states that positions sent to it are "irrecoverable by design", with no owner, no operator, and no code path that transfers a position back out. Fee distribution is driven by `collectFees`, which calls `_collect` to realize a position's accrued fees through a zero-liquidity `Actions.DECREASE_LIQUIDITY` call, followed by an `Actions.TAKE_PAIR` withdrawal.

However, `_collect` hardcodes the `hookData` argument of `DECREASE_LIQUIDITY` to `bytes("")`, and `collectFees` exposes no way to supply alternative data for a given position. In Uniswap v4, `hookData` is forwarded into the pool's hook callbacks on every liquidity modification, including a fee-realization call with a zero `liquidityDelta`. A hook that requires non-empty `hookData` would therefore cause `modifyLiquidities` to revert unconditionally for that position. `FeeSplitter` also cannot gate which positions it ends up custodying: `onERC721Received` accepts any inbound transfer unconditionally, and the `PositionManager` mint path does not invoke it at all, so a hook-restricted position can end up under this custodian without any indication that it is unfit for it. Combined with the irrecoverable custody model, fee collection for that position becomes permanently impossible, and its accrued and future fees are stuck indefinitely.

Consider extending fee collection to accept per-position `hookData`, for example, via `collectFees(CollectParams[])` where each entry carries a `tokenId` and its `hookData`, and forwarding it into `Actions.DECREASE_LIQUIDITY`. Consider also explicitly deciding whether hook-restricted pools are supported and, if not, reverting early in `_collect` with a dedicated error, rather than silently accepting a position that can become permanently fee-locked.

This issue belongs to the first phase of the audit and was identified in the `100fe70a28de1648e51b1a9793be7c0fdd3ae2d7` commit version.

Update: Acknowledged, not resolved.

L-04 Documented ERC-165 Callback Gate Does Not Exist

The [claim documentation](#) of the [IClaimableRecipient](#) interface states that callers declaring [IClaimExecutor](#) support via ERC-165 are invoked through [onClaimed](#), and that other callers are not called back. However, no [supportsInterface](#) check exists anywhere in this path. The callback is instead determined statically by inheritance: recipients extending [BaseClaimRecipient](#) leave [_afterClaim](#) empty and never call back, whereas recipients extending [BaseClaimRecipientWithCallback](#) [invoke onClaimed unconditionally](#). The [IClaimExecutor](#) interface [describes the actual behavior correctly](#), so the two interfaces contradict each other.

The incorrect documentation understates a hard requirement. An integrator relying on it could conclude that claiming from a callback recipient is safe without implementing [onClaimed](#), when in fact every such claim reverts.

Consider correcting the [claim](#) documentation to describe the inheritance-based mechanism, matching the description already given in [IClaimExecutor](#).

This issue belongs to the second phase of the audit.

Update: Resolved in [pull request #218](#).

L-05 Positions Launched Through a Forwarder Can Have Their Beneficiary NFT Claimed by Anyone

The graffiti a UERC20 exposes is a sound identity credential. For a token created through the factory, [BaseUERC20.graffiti](#) is immutable, and the value is part of the CREATE2 salt computed in [UERC20Factory.createToken](#), so it is cryptographically bound to the token's own address and cannot vary between reads or between callers. Minting a position's beneficiary NFT on a graffiti proof should therefore reach the account that actually launched the token.

However, [LiquidityLauncher.createToken](#) stamps [getGraffiti](#) on raw [msg.sender](#), so a token launched through a shared arbitrary-call forwarder records that forwarder as creator of record. [test_e2e_launchAndBuyInOneTransaction_throughMulticall3](#) documents this for Multicall3 and presents that route as the only way to launch and buy in one transaction today, so the affected creators are the ones following the flow the tests demonstrate. For an unregistered position pairing such a token, the credential check in [beforeClaimTransfer](#) can only ever match the

forwarder, and a forwarder by construction executes arbitrary calls on behalf of any account. Batching `claim(tokenId, 0, 0)` followed by `transferFrom(forwarder, attacker, tokenId)` through it makes the forwarder `msg.sender` for both, so the graffiti comparison mints the NFT and the ERC-721 owner check on the transfer passes. Because `_payout` returns early on a zero amount, no funds move during the claim and the forwarder's inability to receive native ETH is never exercised. The attacker ends the transaction holding the NFT that [BeneficiaryVault.beforeClaimTransfer](#) pays every subsequent claim to, and the outcome is permanent: [registerBeneficiary](#) can re-mint only for the position's owner in the PositionManager, which is the splitter. Recovery is therefore a permissionless race in which the original creator holds no privileged position. Positions custodied by [LBPStrategy](#) are the affected population, since it never registers a beneficiary, whereas [InstantLaunchStrategy](#) does. The vault's [documentation](#) notes that an aggregator becomes the prover, but records it as a property of the proof rather than as a third party being able to take the position outright.

Separately, the same authorization decision trusts a value it never authenticates. `_graffitiOf` staticcalls `graffiti()` on a pool currency without establishing that the currency was deployed by the factory, so any contract answering the selector supplies the credential and the returned value is required to be neither immutable nor caller-independent; since `ORIGIN` is readable inside a staticcall, a currency returning `keccak256(abi.encode(tx.origin))` authorizes whoever is asking. This is not a theft path against an honest creator, because the only contract consulted is the token that creator paired themselves, and a creator who deliberately paired a lying token gains nothing a factory token would not already grant them. It matters instead because the absence of a local provenance check leaves the vault's safety resting on invariants enforced two contracts away and invisible from the call site: [FeeSplitter.collect](#) reverts unless `currency0` is native ETH and `_graffitiOf` reports `bytes32(0)` for native, so only `currency1` can supply a credential at all, and [FeeSplitter.distribute](#) keeps funds and attribution in one call under the shipped [split configuration](#), so `_attribute` leaves no unattributed surplus for a forged credential to consume. [_validateAndStoreSplits](#) does not reject a claim-recipient split that leaves `useCallback` unset, a configuration that would transfer to the vault indefinitely without attributing and manufacture exactly the surplus the check currently denies.

Consider having `createToken` accept an explicit creator address rather than stamping raw `msg.sender`, so a caller routing through a forwarder can name the account that should hold the credential.

Update: Acknowledged, not resolved. The team stated:

| Ack and accepted / documented

L-06 Unverified External Return Value Authorizes Beneficiary NFT Claims

The graffiti a UERC20 exposes is a sound identity credential. For a token created through the factory, [BaseUERC20.graffiti](#) is immutable, and the value is part of the CREATE2 salt computed in [UERC20Factory.createToken](#), so it is cryptographically bound to the token's own address and cannot vary between reads or between callers. A contract that authorizes on graffiti can verify that binding locally before trusting it.

However, [_graffitiOf](#) staticcalls [graffiti\(\)](#) on a pool currency and passes the result directly into the authorization decision in [_beforeClaimTransfer](#), which mints the beneficiary NFT to [msg.sender](#) whenever [keccak256\(abi.encode\(msg.sender\)\)](#) matches either side. The vault never establishes that the currency was deployed by the factory, so any contract answering the selector supplies the credential, and the returned value is required to be neither immutable nor caller-independent. Since [ORIGIN](#) is readable inside a staticcall, a currency returning [keccak256\(abi.encode\(tx.origin\)\)](#) authorizes whoever is asking, and two unrelated callers can each pass the creator proof for the same unregistered position and take the attributed funds.

The reason this does not currently yield a theft primitive is enforced outside the vault.

[FeeSplitter.collect](#) reverts unless [currency0](#) is native ETH, and [_graffitiOf](#) reports [bytes32\(0\)](#) for native, so only [currency1](#) can ever supply a credential.

[FeeSplitter.distribute](#) transfers a split share unconditionally but calls [onAmountsReceived](#) only when [split.useCallback](#) is set, so under the shipped [split configuration](#) the funds and their attribution land in the same call. [BaseClaimRecipient.attribute](#) then reverts unless the balance already covers [totalAmounts](#) plus the new amount, leaving no unattributed surplus for a forged credential to consume. None of these guarantees is visible from the call site that depends on them, and [_validateAndStoreSplits](#) does not reject a claim-recipient split that leaves [useCallback](#) unset, a configuration that would transfer to the vault indefinitely without attributing and manufacture exactly the surplus required.

The same root cause has a second consequence. [LiquidityLauncher.getGraffiti](#) derives the value from raw [msg.sender](#) in [LiquidityLauncher.createToken](#), so a token launched through a shared arbitrary-call forwarder records that forwarder as creator of record, as [test_e2e_launchAndBuyInOneTransaction_throughMulticall3](#) documents for Multicall3. For an unregistered position pairing such a token the NFT can only ever mint to the forwarder, and because [_payout](#) returns early on a zero amount, a zero-amount claim batched with a [transferFrom](#) routed through the forwarder extracts it. Recovery therefore becomes a permissionless race in which the original creator holds no privileged position. Positions

custodied by [LBPStrategy](#) are the affected population, since it never registers a beneficiary, whereas [InstantLaunchStrategy](#) does.

Consider verifying provenance locally rather than trusting the returned value, by recomputing the factory CREATE2 address from the token metadata and the returned graffiti and requiring it to equal the currency address, which makes the credential self-authenticating. Consider also having `createToken` accept an explicit creator address rather than stamping raw `msg.sender`, and having `_validateAndStoreSplits` reject a claim-recipient split that leaves `useCallback` unset. Note that requiring `msg.sender == tx.origin` does not address this, since it breaks smart accounts and a caller-dependent graffiti value defeats it directly.

Update: *Acknowledged, not resolved. The team stated:*

| *Ack, not fixed*

L-07 Oversized Graffiti Response Permanently Blocks Claims For Unregistered Positions

The vault reads at most a single 32-byte word from a pool currency's `graffiti()`, and a currency that answers with anything else is discarded at no cost, so an unregistered position's attributed fees stay claimable.

However, `_graffitiOf` binds the `staticcall` result to `bytes memory`, so the generated decoder allocates `returndatasize()` bytes and copies the entire payload into memory before the `data.length == 32` guard is evaluated. A currency that returns a multi-megabyte payload spends the 63/64 of gas forwarded to it building the reply, leaving the vault the remaining 1/64 to copy it, and `claim` reverts out of gas. Raising the gas budget does not help, because the payload is sized from the gas the callee receives: a proof of concept reverts at 10, 30, 100, and 500 million gas. The read is reached from `_beforeClaimTransfer` whenever `_ownerOf(tokenId)` is zero, which runs inside `claim` before any payout, so every claim for that position reverts. For positions custodied by a `FeeSplitter` this is permanent, since `registerBeneficiary` requires position custody and the splitter never transfers positions out, leaving both attributed amounts unclaimable with no rescue path. Entry is permissionless through `onERC721Received` and the unauthenticated `onAmountsReceived`. Note that a currency that merely burns the forwarded gas and reverts is harmless, since `success` is then false and the claim proceeds to the fallback flush.

Consider reading the graffiti with inline assembly that rejects `returndatasize() != 32` before copying and writes at most 32 bytes into scratch space, so an oversized reply is discarded without ever entering memory.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged

L-08 Prefunded Router Inputs Can Be Stranded and Swept by Third Parties

When distributing an ERC-20, the `initializeDistribution` entry point transfers the full `amount` to the router before running the route, then refunds only `the strategy's own leftover balance` to `config.recipient`. That refund captures tokens the route sent back to the strategy, but never tokens left on the router, because the router holds them under its own address where `IERC20(token).balanceOf(address(this))` cannot see them. If a route spends less than `amount` and omits a sweep to a trusted recipient, the remainder stays in the router's global balance, and a later router call can move it to any recipient. The `native path` carries the same exposure with no recovery at all, since the strategy `has no receive function` and cannot take native back.

Consider, as a lightweight fix, adding a post-execution check that the router's balance of the input asset is not greater after `execute` than before the prefunding transfer, and reverting otherwise, so that an underspending route cannot silently leave inputs on the router. Consider, as a structural alternative for the token path, replacing the up-front transfer with a Permit2 allowance to the router so the route can pull only the amount each leg actually spends, which removes the leftover rather than detecting it.

Update: Acknowledged, not resolved.

Notes & Additional Information

N-01 Unsanitized `creator()` Results Can Strand or Burn the `CREATOR` Share

`FeeSplitter` supports a `CREATOR_SENTINEL` recipient that resolves per pool to `currency1.creator()`, substituted for the sentinel inside `_distribute`. The resolved creator comes from `_resolveCreator` and is used in `_collect`.

However, the result of `_resolveCreator` is only sanitized against `address(0)`, never against `address(this)` or `CREATOR_SENTINEL`. If a pool's `currency1.creator()` resolves to `address(this)`, that value is passed straight through as `creatorRecipient`, and `_transfer` then attempts to send the `CREATOR` share to the splitter itself. For the native ETH side, this self-directed call succeeds trivially through the contract's own `receive` function without actually reducing `address(this).balance`, so the payout never leaves the contract. This breaks the "every collect ends at zero" invariant [described in _collect's own comment](#) and strands that value in the contract's standing balance. Because `_collect` distributes the contract's entire standing balance on every call rather than a per-call delta, this stranded residual is later picked up and redistributed during an unrelated `collectFees` call for a different `tokenId`, misattributing one pool's creator share to another.

Separately, if `creator()` resolves to `CREATOR_SENTINEL` itself, the substitution `recipient = creatorRecipient` performed by `_distribute` becomes a no-op and `recipient` stays equal to `CREATOR_SENTINEL`, which is an address derived from a hash that nobody controls. `_transfer` then sends the `CREATOR` share there, permanently burning it.

Consider sanitizing the resolved creator before it is used for distribution, treating `address(0)`, `address(this)`, and `CREATOR_SENTINEL` as unresolvable and redirecting those shares to the appropriate per-side fallback, mirroring the `_isValidFeeRecipient` check already applied elsewhere in the contract.

This issue belongs to the first phase of the audit and was identified in the [100fe70a28de1648e51b1a9793be7c0fdd3ae2d7](#) commit version.

Update: Acknowledged, not resolved. The team stated:

N-02 Incorrect and Incomplete Documentation

Throughout the codebase, the following instances of incorrect or incomplete documentation were identified:

- In `IFeeSplitter.sol`, the NatSpec for `FEE_BENEFICIARY_SENTINEL` states that it resolves to the position's registered fee beneficiary, falling back to the pool token's UERC20 `creator()` when no beneficiary was registered. However, `FeeSplitter` never queries `creator()` anywhere in its implementation. When `feeBeneficiary[tokenId]` is unset, `_collect` resolves the beneficiary argument straight to the per-side fallback addresses `nativeFallback` and `tokenFallback`, and `_distribute` substitutes that same fallback wherever a split names `FEE_BENEFICIARY_SENTINEL`. The documented fallback behavior does not match the implementation.
- In `FeeSplitter.sol`, the `onERC721Received` function has an incomplete docstring: none of its parameters, including `tokenId` and `data`, are documented, and neither is its `bytes4` return value.

Consider aligning the `FEE_BENEFICIARY_SENTINEL` documentation with its actual behavior, either by implementing the documented `creator()` fallback when `feeBeneficiary[tokenId]` is unset, or by updating the NatSpec to state plainly that unregistered positions fall back to `nativeFallback` and `tokenFallback`. Consider also thoroughly documenting the parameters and return value of `onERC721Received`, following the [Ethereum Natural Specification Format](#) (NatSpec).

The first two instances belong to the first phase of the audit and were identified in the `607e3faa432fd505083452f7a2b3682fbfa8a7be` commit version. The last instance belongs to the second phase of the audit.

Update: Resolved in pull requests [#201](#), [#205](#) and [#210](#).

N-03 Floating Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled.

The `FeeSplitter.sol` file has the `solidity ^0.8.26` floating pragma directive.

Consider using fixed pragma directives.

This issue belongs to the first phase of the audit and was identified in the [607e3faa432fd505083452f7a2b3682fbfa8a7be](#) commit version.

Update: Acknowledged, not resolved.

N-04 Unused Import

In [FeeSplitter.sol](#), the [FullMath](#) library is imported but never used.

Consider removing this unused import to improve the overall clarity and readability of the codebase.

This issue belongs to the second phase of the audit.

Update: Resolved in [pull request #218](#).

N-05 Functions Updating State Without Event Emissions

Throughout the codebase, a single instance of a function updating state without an accompanying event emission was identified:

- The [onAmountsReceived](#) function in [BaseClaimRecipient.sol](#)

Consider emitting events whenever there are state changes to improve transparency and make the codebase less error-prone.

This issue belongs to the second phase of the audit.

Update: Resolved in [pull request #218](#).

N-06 Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- In [BaseClaimRecipient.sol](#), the [Amounts](#) struct
- In [BaseClaimRecipient.sol](#), the [amounts](#) state variable
- In [BaseClaimRecipient.sol](#), the [totalAmounts](#) state variable

- In `FeeSplitter.sol`, the `_splits` state variable

Consider thoroughly documenting all functions and state variables (and, where applicable, their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

This issue belongs to the second phase of the audit.

Update: Resolved in [pull request #218](#).

N-07 Incomplete Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- In `IBeneficiaryVault.sol`, the `nativeFallback` function is missing the return values.
- In `IBeneficiaryVault.sol`, the `tokenFallback` function is missing the return values.
- In `IClaimableRecipient.sol`, the `InvalidPosition` error is missing the `tokenId` parameter, the `InsufficientAmountReceived` error is missing the `currency`, `received`, and `expected` parameters, and the `InvalidTransferRecipient` error is missing the `currency` parameter.
- In `IClaimableRecipient.sol`, the `Claimed` event is missing the `tokenId`, `currency0Amount`, `currency1Amount`, and `poolKey` parameters.
- In `IClaimableRecipient.sol`, the `amounts` function is missing the `tokenId` parameter and the return values.
- In `IFeeSplitter.sol`, the `BalanceExceedsMaxAllowed` error is missing the `tokenId` parameter.
- In `IFeeSplitter.sol`, the `positionManager` function is missing the return values.
- In `IFeeSplitter.sol`, the `getSplits` function is missing the return values.
- In `BaseClaimRecipient.sol`, the `_beforeClaimTransfer` function is missing all of its parameters and return values.
- In `BaseClaimRecipient.sol`, the `_payout` and `_attribute` functions are missing their parameters.
- In `BaseClaimRecipientWithCallback.sol`, the `_beforeExecutorCallback` and `_afterExecutorCallback` functions are both missing the `_poolKey` and `_tokenId` parameters.

- In `CompoundingClaimRecipient.sol`, the `NotEnoughLiquidityAdded` error is missing the `required` and `actual` parameters.
- In `FeeSplitter.sol`, the `onERC721Received` function is missing the parameters and the return values.
- In `FeeSplitter.sol`, the `_requireNoPendingFees` and `_distribute` functions are missing their parameters.
- In `DirectLaunchStrategy.sol`, the `_pull` function is missing its parameters and the return value.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec). Note that the `_beforeClaimTransfer`, `_beforeExecutorCallback`, and `_afterExecutorCallback` hooks deserve particular attention: they form the extension surface that any future claim recipient overrides, and the neighboring `_afterClaim` hook already documents all of its parameters, so the omission is inconsistent within a single file.

This issue belongs to the second phase of the audit.

Update: Resolved in [pull request #218](#).

N-08 Coding Inconsistencies

Throughout the codebase, multiple inconsistencies were identified:

- The `increaseLiquidity` function of the `FeeSplitter` contract branches on the result of `poolManager.isUnlocked` and dispatches its actions through `modifyLiquiditiesWithoutUnlock` whenever the caller already holds the lock. Fee collection makes no such distinction: the `_collect` function, on which `collectFees` relies to realize a position's fees, unconditionally calls `modifyLiquidities`. As a result, `collectFees` reverts with `AlreadyUnlocked` in precisely the composition contexts that `increaseLiquidity` was written to support, which limits the composability of the splitter for integrations that batch pool actions within an existing unlock.
- The `constructor` of the `DirectLaunchStrategy` contract rejects a fee splitter that is not bound to the same `PositionManager` as the strategy, and it [further verifies](#) that the configured beneficiary vault is a callback recipient of that splitter. However, no equivalent assertion is made on the beneficiary vault's own `PositionManager`, even though [registration](#) is authorized by proving position custody against it. A vault bound to

a different `PositionManager` consequently passes deployment and then reverts every launch performed through that strategy instance, leaving the last leg of an otherwise complete deploy-time wiring check unverified.

- The `MIN_LIQUIDITY_INCREASE` variable of the `CompoundingClaimRecipient` contract is the only immutable in the reviewed files that is named in upper snake case. Every other immutable, including `positionManager`, `nativeFallback`, `tokenFallback`, `poolManager`, `launcher`, `feeSplitter`, `beneficiaryVault`, `initialTick`, `initialSqrtPriceX96`, and `positionLiquidity`, uses lower camel case, while upper snake case is otherwise reserved for `constant` declarations.

Consider reverting with a dedicated error in `_collect` when the `PoolManager` is already unlocked, so that both entry points behave predictably under the same conditions and the failure is self-describing rather than surfacing as a `PoolManager` error. Note that the apparent alternative, mirroring the lock-state branch of `increaseLiquidity` inside `_collect`, is not equivalent in effect and should be approached with care: `_collect` currently opens and closes its own lock before returning, so `_distribute` and every `onAmountsReceived` callback run with the `PoolManager` locked. Dispatching through `modifyLiquiditiesWithoutUnlock` would move the whole distribution inside the caller's lock, so a split recipient that itself needs to open one would begin reverting depending on who triggered the collection, and every recipient would gain a window with an unlocked `PoolManager` while holding funds that have just been pushed to it. If the composition path is intended, it is better placed behind a separate entry point that documents this difference. In addition, consider exposing the `positionManager` getter, which `BaseClaimRecipient` already declares as `public`, through the `IBeneficiaryVault` interface, and asserting it in the `DirectLaunchStrategy` constructor alongside the existing wiring checks. Finally, consider aligning the naming of `MIN_LIQUIDITY_INCREASE` with the convention used by the other immutable variables. Applying the same conventions at every equivalent site improves the clarity and consistency of the codebase.

This issue belongs to the second phase of the audit.

Update: Resolved in [pull request #218](#).

N-09 Unused Named Return Variables

Named return variables are a way to declare variables that are meant to be used within a function's body for the purpose of being returned as that function's output. They are an alternative to explicit in-line `return` statements.

Throughout the codebase, multiple instances of unused named return variables were identified:

- In `BaseClaimRecipient.sol`, the `recipient0`, `toSend0`, `recipient1`, and `toSend1` return variables of the `_beforeClaimTransfer` function
- In `BeneficiaryVault.sol`, the `recipient0`, `toSend0`, `recipient1`, and `toSend1` return variables of the `_beforeClaimTransfer` function

Consider either using or removing any unused named return variables.

This issue belongs to the second phase of the audit.

Update: Resolved in [pull request #218](#).

Client Reported

CR-01 Permissionless Pool Initialization Allows Permanently Bricking a Direct Launch

The `initializeDistribution` function of `DirectLaunchStrategy` builds a fully deterministic `PoolKey` for the launch pool: `currency0` is always the native currency, `currency1` is the launched `token`, and `fee`, `tickSpacing`, and `hooks` are fixed constants, with `hooks` set to `address(0)`. The `token` address itself is a `CREATE2` address computed by `UERC20Factory` from `name`, `symbol`, `decimals`, the fixed `LiquidityLauncher` address, and a graffiti value derived from the original caller. All of these inputs are public or predictable before the launch transaction lands, and the resulting address can be recomputed off-chain through the `getUERC20Address` view function of the factory. Consequently, the entire `PoolKey` used can be reconstructed by any observer before `distributeToken` is executed, and in most cases even before `createToken` is executed.

Because the pool carries no hooks, `PoolManager.initialize` can be called permissionlessly by anyone holding the precomputed key, at the cost of a single inexpensive transaction, and will succeed regardless of whether the `currency1` address currently holds contract code. Once a pool for a given key is initialized, any later call to `initialize` for the same key reverts with `PoolAlreadyInitialized` in `Pool.sol`.

This allows any third party to permanently deny a specific token's launch through `DirectLaunchStrategy` at negligible cost and without any privileged access. The target token address can be derived as soon as the creator's identity and the intended `name`,

`symbol`, and `decimals` values are known, or observed in the mempool of the `createToken` transaction. The attack can therefore be carried out proactively, well before `distributeToken` is submitted, so batching `createToken` and `distributeToken` into a single `Multicall` transaction does not close the window. Once the pool has been initialized by a third party, the affected token can never be launched through this strategy, and the only recourse is deploying an entirely new token under different parameters, discarding any supply, holders, or announcements already associated with the bricked one.

Consider attaching the minimal `InitializerHook`, authorized to `DirectLaunchStrategy` itself, to the launch pool's `PoolKey`, so that only the strategy can successfully initialize the pool and arbitrary initialization is no longer possible.

Update: Acknowledged, not resolved. The team stated:

On frontrunning, I don't see a good solution besides adding a hook which is a nonstarter atm.

CR-02 Creator Fee Share Silently Misdirected to LiquidityLauncher on Every Launch

`LiquidityLauncher`'s `createToken` forwards every token creation to the configured `ITokenFactory`, including the canonical `UERC20Factory`. This holds whether the end user calls `createToken` directly, through `LiquidityLauncher.multicall`, or via an external aggregator such as `Multicall3`, since the final call into the factory is always made by `LiquidityLauncher`'s own code. The `CREATOR_SENTINEL` mechanic of `FeeSplitter` is built to route a configured share of every collected fee to a launched token's registered creator, resolved through a `staticcall` to the `creator` accessor of that token.

However, `UERC20Factory.createToken` records the deployed token's immutable `creator` as `msg.sender` at creation time. Because `LiquidityLauncher` is the caller of the factory, and `ITokenFactory.createToken` exposes no parameter for a creator distinct from `msg.sender`, every token created through the intended entry point ends up with `creator() == address(LiquidityLauncher)`, never the address of the user who actually initiated the launch.

Consequently, on every `collectFees` call for every standard launch, `_resolveCreator` resolves to `address(LiquidityLauncher)`, and the `CREATOR_SENTINEL` share of both currency sides is distributed to that address. `LiquidityLauncher` defines no `receive` or payable `fallback` function, and none of its `Multicall` or `Permit2Forwarder` bases define one either. Its entire external surface is `createToken`, `depositToken`,

`distributeToken`, and `getGraffiti`, plus the inherited `multicall` and `permit` functions, none of which can move out an arbitrary ETH or token balance.

As a result, [the native ETH creator share fails to reach `LiquidityLauncher` and is silently redirected to the configured `nativeFallback` instead](#), with no revert or signal that the payout went anywhere other than the intended creator. The token-side creator share, by contrast, is delivered through a plain ERC-20 transfer that needs no receive hook and therefore succeeds, permanently trapping those tokens inside `LiquidityLauncher` with no rescue path anywhere in the codebase. The `CREATOR_SENTINEL` mechanic is therefore defeated on every launch that goes through the documented entry point.

Consider updating the token creation flow so a launched token's registered `creator` reflects the address that actually initiated the launch rather than `LiquidityLauncher`, for example by extending `ITokenFactory.createToken` with an explicit creator parameter distinct from `msg.sender`, and consider adding a mechanism to recover the ETH and token amounts already misdirected to `LiquidityLauncher`.

Update: Resolved in [pull request #199](#).

Conclusion

The Liquidity Launcher system provides token launch and liquidity bootstrapping on top of Uniswap v4. OpenZeppelin audited its direct launch path over three phases. The first phase covered the strategy that opens a pool and places a token's full supply as a single liquidity position, together with the splitter that takes permanent custody of that position and distributes the fees it earns. The second phase covered the reworked fee-distribution design that followed, in which per-position attribution moved out of the splitter and into a separate family of claim recipients. The third phase covered a scope extension to that design, in which the launch strategy was renamed and its beneficiary vault made optional, and a new vault variant let the creator of a launcher-created token claim an unregistered position's beneficiary share by proving authorship through the token's on-chain graffiti.

The rework narrows the splitter's responsibilities and places beneficiary resolution behind an explicit registration step authorized by position custody, removing the untrusted per-position resolution path that the first phase found could be redirected by the launched token itself. At the same time, it introduces a callback surface spanning the splitter, the claim recipients, and the callers that drive claims, where obligations are enforced by post-conditions checked across a callback rather than by the identity of the caller. Several of the second phase's findings relate to that surface and to the documentation describing it. The interface documentation states an ERC-165 gate on the claim callback that the implementation does not perform, which an integrator could rely on to its detriment. In addition, the splitter's two permissionless entry points disagree on whether they tolerate an already-unlocked `PoolManager`, and the deployment-time wiring checks stop short of the vault's own `PositionManager` binding. Both phases also identified instances in which inline documentation, docstrings, and event coverage lag behind the implementation, which is worth reconciling before integrators rely on the system.

Because the splits, fallback addresses, price band, and contract wiring cannot be changed after deployment, and because positions are irrecoverable by design once transferred to the splitter, any state that a permissionless caller can establish against a launched pool is effectively permanent. The second phase identified that any account can saturate the launch position's boundary tick at negligible cost and thereby prevent every subsequent liquidity increase on that position, which permanently freezes the fees already attributed to a `CompoundingClaimRecipient` with no recovery path for the affected parties. Validating deployment parameters, confirming that the strategy, the splitter, and the vault are bound to

the same `PositionManager`, and deliberately choosing which recipients appear in an instance's splits therefore remain the most consequential decisions for anyone relying on the system.

OpenZeppelin thanks the Uniswap team for their cooperation and responsiveness throughout this engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.